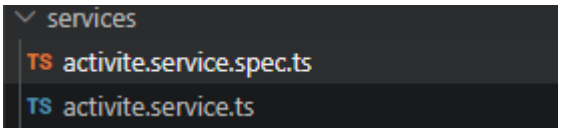
ICOM - MASTER
CIM - TD
GHIZLAN Moqim

1. Partie I: Services
   1.1 Depuis un terminal placé à la racine du projet, générez un service nommé ActiviteService dans le répertoire src/app/services à l'aide de la commande suivante

```
ng generate service services/activite
```



activite.service.spec.ts

```ts
import { TestBed } from '@angular/core/testing';
import { ActiviteService } from './activite.service';

describe('ActiviteService', () => {
  let service: ActiviteService;

  beforeEach(() => {
    TestBed.configureTestingModule({});
    service = TestBed.inject(ActiviteService);
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });
});
```

activite.service.ts

```ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root',
})
export class ActiviteService {}
```

1.2 Dans la classe HomeComponent, injectez dans le constructeur le service ActiviteService en le déclarant comme paramètre et en important correctement la classe.

home.ts

```
export class Home {
  constructor(
    private monRouteur: Router,
    private activiteService: ActiviteService,
  ) {}
```

1.3 Déplacez le tableau objectifs de la classe HomeComponent vers la classe ActiviteService, puis mettez à jour tous les composants qui utilisent ce tableau (y compris dans les templates HTML). Tous les composants doivent accéder au même tableau fourni par le service ; ne créez pas une copie indépendante dans chaque composant. Les animations doivent toujours fonctionner.

On ajoute **activiteService** dans le constructeur de Home

home.ts

```
constructor(
    private monRouteur: Router,
    readonly activiteService: ActiviteService,
  ) {}
```

On ajoute aussi les getteurs

home.ts

```
get objectifs(): string[] {
    return this.activiteService.objectifs;
  }

  set objectifs(objectifs: string[]) {
    this.activiteService.objectifs = objectifs;
  }

  get nbItems(): number {
    return this.activiteService.objectifs.length;
  }
```

On modifie l'**ajouteItem, modifierItem et supprimerItem**

home.ts

```ts
ajoutItem(): void {
    const objectif = this.objectifText.trim();
    if (!objectif) {
      return;
    }
    if (this.indexModification === null) {
      this.activiteService.objectifs.push(objectif);
      setTimeout(() => {
        this.monRouteur.navigate(['/about']);
      }, 1000);
    } else {
      this.activiteService.objectifs[this.indexModification] = objectif;
    }
    this.annulerModification();
  }
```

home.ts

```ts
supprimerItem(index: number): void {
    this.activiteService.objectifs.splice(index, 1);
    if (this.indexModification === index) {
      this.annulerModification();
    } else if (this.indexModification !== null && index < this.indexModification) {
      this.indexModification--;
    }
  }
```

home.ts

```ts
modifierItem(index: number): void {
    this.indexModification = index;
    this.objectifText = this.activiteService.objectifs[index];
  }
```

Et on modifie **home.html** afin d'utiliser le service

home.html

```
@for (objectif of activiteService.objectifs; track $index) {
```

1.4 Vérifiez que les composants s'affichent correctement, même après navigation entre HomeComponent et AboutComponent. Vérifiez également qu'une modification effectuée dans un composant reste visible après la navigation.

On prépare la classe **About**

about.ts

```ts
export class About {
  readonly activiteService: ActiviteService;

  constructor(activiteService: ActiviteService) {
    this.activiteService = activiteService;
  }
}
```

Et on finalise la page **about.html**

about.html

```html
<h2>Activités partagées</h2>

@for (objectif of activiteService.objectifs; track $index) {
  <p class="life-container" animate.enter="anim">{{ objectif }}</p>
} @empty {
  <p>Aucune activité enregistrée.</p>
}
```

1.5 Reprenez la question 4.3 du TP précédent, mais appliquez cette fois les modifications à la classe AboutComponent. Affichez ensuite sous forme d'items la liste des éléments du tableau objectifs.
Vous pourrez obtenir un affichage similaire à :

On apporte les modification sur **about.html**

about.html

```
<p>about works!</p>
<ol>
@for (objectif of activiteService.objectifs; track $index) {
  <li animate.enter="anim">{{ objectif }}</li>
} @empty {
  <li>Aucune activité enregistrée.</li>
}
</ol>
```

1.6 Dans le fichier home.component.html, lors de l'affichage des différentes activités, ajoutez un événement souris permettant d'ouvrir la page consultAct en ajoutant à l'URL l'indice de l'activité sélectionnée.
Indication : avec la syntaxe de contrôle @for, l'indice de l'élément parcouru est accessible avec la variable $index. Utilisez cet indice pour construire l'URL. Vérifiez que la route correspondante est bien définie pour recevoir cet indice.

home.html

```
<button type="button" class="life-container"
            animate.enter="anim" [class.editing]="indexModification === $index"
            [attr.aria-pressed]="indexModification === $index"
            (click)="consulterActivite($index)">
```

home.html

```
<button type="button" class="btn secondary"
            (click)="modifierItem($index)">Modifier</button>
```

1.7 Dans consult-activite.component.ts, injectez le service ActiviteService. Récupérez ensuite l'indice transmis dans l'URL à l'aide de ActivatedRoute et utilisez-le pour retrouver l'activité correspondante dans le tableau du service. Mettez à jour consult-activite.component.html

On apporte les modifications sur **consult-activite.ts** :

consult-activite.ts

```ts
export class ConsultActivite {
  readonly activite: string | undefined;

  constructor(
    private route: ActivatedRoute,
    private activiteService: ActiviteService,
  ) {
    const index = Number(this.route.snapshot.params['id']);
    this.activite = Number.isInteger(index) && index >= 0
      ? this.activiteService.objectifs[index]
      : undefined;
  }
}
```

Et ensuite on applique notre affichage avec une vérification sur **consult-activite.html** :

consult-activite.html

```html
<h3>Détail de l'activité :</h3>
<ol>
  @if (activite !== undefined) {
    <li>Nom de l'activité : <strong>{{ activite }}</strong></li>
  } @else {
    <li>Activité introuvable.</li>
  }
</ol>
```

1.8 On souhaite ajouter la possibilité de supprimer un élément de la liste des activités depuis l'onglet « détails ». Pour cela :

- Comme la classe ActiviteService centralise le tableau, ajoutez une fonction removeItem(index: number) qui supprime l'élément du tableau (utilisez splice).

- Dans ConsultActiviteComponent, ajoutez une fonction supprimeElem() prenant en paramètre l'indice de l'élément à supprimer.
- Ajoutez un bouton permettant d'appeler cette fonction
- Rediriger ensuite vers la page principale (dans supprimeElem()).

On commence par ajouter removeItem dans le service activite

activite.service.ts

```
removeItem(index: number): void {
    this.objectifs.splice(index, 1);
  }
```

Ensuite on aure besoin d'un variable Index afin de stoker l'ID dans consult-activite

consult-activite.ts

```
readonly index: number;
```

Ensuite on modifie le constructeur de consult-activite afin d'ajouter le router

consult-activite.ts

```
private router: Router,
```

Ensuite on applique notre logique pour récupérer l'ID et on fait la vérification nécessaire

consult-activite.ts

```
const routeIndex = Number(this.route.snapshot.params['id']);
    this.index = Number.isInteger(routeIndex) && routeIndex >= 0 ? routeIndex : -1;
    this.activite = this.index >= 0
      ? this.activiteService.objectifs[this.index]
      : undefined;
  }
```

Ensuite on implémente la fonction supprimElem dans consult-activite.ts

consult-activite.ts

```ts
supprimeElem(index: number): void {
    this.activiteService.removeItem(index);
    this.router.navigate(['/']);
  }
```

Et enfin on ajoute le bouton dans consult-activite.html

consult-activite.html

```html
<li>
      <button
          type="button"
          class="btn delete"
          (click)="supprimeElem(index)">Supprimer
      </button>
    </li>
```

1.9 Enfin, on souhaite pouvoir modifier le titre d'une activité. En vous inspirant de la question précédente, ajoutez la fonctionnalité d'édition et de sauvegarde du titre d'une activité.

En appliquant les même modification que dans l'exo passé
On commence par ajouter la fonction updateItem dans notre service

activite.service.ts

```ts
updateItem(index: number, titre: string): void {
    this.objectifs[index] = titre;
  }
```

Ensuite on ajoute les variables nécessaires pour notre logique d'update dans CA

consult-activite.ts

```ts
export class ConsultActivite {
  readonly index: number;
  activite: string | undefined;
  titreModifie = '';
  editionEnCours = false;
```

Une petite vérification dans le constricteur de CA

consult-activite.ts

```ts
this.titreModifie = this.activite ?? '';
```

Et on termine par ajouter les fonctions demandées :

consult-activite.ts

```ts
modifierTitre(): void {
    if (this.activite !== undefined) {
      this.titreModifie = this.activite;
      this.editionEnCours = true;
    }
  }

  sauvegarderTitre(): void {
    const titre = this.titreModifie.trim();
    if (!titre || this.activite === undefined) {
      return;
    }

    this.activiteService.updateItem(this.index, titre);
    this.activite = titre;
    this.titreModifie = titre;
    this.editionEnCours = false;
  }

  retour(): void {
    this.router.navigate(['/']);
  }
```

2. Partie 2: Déploiement

2.1 Créez sur GitHub un nouveau dépôt vide qui servira à publier votre application. Choisissez un nom simple, sans espace, puis notez le nom du dépôt et votre nom de compte GitHub.

https://github.com/moqim-ghizlan/angular-tasks

2.2 Associez le projet local au dépôt GitHub. Remplacez les deux éléments en majuscules par les informations de votre compte et de votre

```
moqim@DESKTOP-SIG7N35 UCRT64 ~/Desktop/cim/Champs d'applications/cours 1/angular-tasks (master)
$ git remote -v
origin  https://github.com/moqim-ghizlan/angular-tasks.git (fetch)
origin  https://github.com/moqim-ghizlan/angular-tasks.git (push)
```

2.3 Installez l'outil de déploiement si nécessaire. Lors du premier lancement de la commande cidessous, Angular peut proposer d'installer le paquet angular-cli-ghpages. Acceptez cette installation si elle est proposée ; sinon, ajoutez le paquet en suivant les indications du terminal.

```
moqim@DESKTOP-SIG7N35 UCRT64 ~/Desktop/cim/Champs d'applications/cours 1/angular-tasks (master)
$ ng add angular-cli-ghpages
✔Determining Package Manager
  › Using package manager: npm
✔Searching for compatible package version
  › Found compatible package version: 3.1.2.
✔Loading package information
✔Confirming installation
✔Installing package

    🚀 angular-cli-ghpages is ready!

    Next steps:
      1. Read the docs: https://github.com/angular-schule/angular-cli-ghpages
      2. Deploy via: ng deploy
      3. Have a nice day!
UPDATE angular.json (2265 bytes)

moqim@DESKTOP-SIG7N35 UCRT64 ~/Desktop/cim/Champs d'applications/cours 1/angular-tasks (master)
$ ng build --base-href=/angular-tasks/
Initial chunk files | Names         |  Raw size | Estimated transfer size
main-3W5LJIDA.js    | main          | 277.58 kB |               73.73 kB
styles-UQSTIDK4.css | styles        |   3.15 kB |              502 bytes

                    | Initial total | 280.73 kB |               74.23 kB

Application bundle generation complete. [12.140 seconds] - 2026-10-16T15:23:15.963Z

Output location: C:\Users\moqim\Desktop\cim\Champs d'applications\cours 1\angular-tasks\dist\Tp3Angular
```

2.4 Ouvrez dans un navigateur l’adresse suivante, en remplaçant les éléments en majuscules : https://VOTRE_COMPTE.github.io/VOTRE_PROJET/.

ça donnera donc :
https://moqim-ghizlan.github.io/angular-tasks/

En conclusion, il apparaît que, dans un environnement local sur http://localhost:4200, les deux liens produisent un résultat identique en dirigeant l'utilisateur vers http://localhost:4200/about. Toutefois, leurs mécanismes de fonctionnement diffèrent. Le lien utilisant href="/about" constitue une balise HTML standard : le navigateur procède à un rechargement complet de la page et sollicite /about depuis la racine du domaine, sans égard pour le base href de l'application. En contexte local, l'application étant située à la racine du serveur, ce lien semble valide. Cependant, lors du déploiement sur GitHub Pages, l'application est hébergée dans le sous-dossier /angular-tasks/ ; le lien href="/about" redirige alors vers https://moqim-ghizlan.github.io/about, emplacement extérieur à l'application, ce qui entraîne une erreur 404. À l'inverse, routerLink="/about" est pris en charge par le routeur Angular, lequel calcule le chemin en tenant compte du base href (/angular-tasks/) et permet une navigation sans rechargement de l'application, aboutissant correctement à https://moqim-ghizlan.github.io/angular-tasks/about. Par conséquent, il est préconisé d'utiliser systématiquement routerLink pour la navigation au sein d'une application Angular.